

CSE 144 Final Project Report
Transfer Learning Challenge
Group Members (max 3):
Robert Schmidling (CruzID: 2246830)
William Zhang (CruzID: 1825257)
Leonard Ifrim (CruzID: 1965894)
Spring 2026

1 Introduction

The problem our group was tasked with solving was training a classifier that efficiently classifies 1000 images in a test set with high accuracy. Transfer learning is appropriate for this task since training a highly complex model would take extensive resources and computation time. Utilizing pre-trained models allowed us to achieve much faster training speeds and higher accuracies. Our approach used multiple preprocessing and data augmentation techniques, DinoV3 for our pretrained weights, and an MLP classifier head. Our final test accuracy was 0.87272.

2 Dataset

The train and test directories include 100 classes. Our train set includes 1079 images, and our test set includes 1036 images. A `CachedViewDataset` class maps each image to an integer label derived directly from its folder name to ensure correct label mapping. Train preprocessing included random resized crop with scale 0.5-1.0, random horizontal flip, color jitter (brightness, contrast, and saturation set to 0.3, hue to 0.1). Random rotation at 15%, Normalization, and random erasing at 0.25.

For validation and test sets, all images were resized to 256x256, they were center cropped to 224, and normalized with the backbone's mean/std pulled with `AutoImageProcessor`. These transformations are called when creating embeddings of the train and test sets, which runs each image through the frozen backbone once to extract token representations. Since the backbone weights are frozen, computing and caching these embeddings means each training step only passes the MLP head forward, making training significantly quicker.

3 Implementation

3.1 Model

We chose Meta DinoV3, `dinov3-vitl16-pretrain-lvd1689m`, as the backbone as it was a cutting edge vision transformer model that had excellent pretrained weights for producing high-quality dense features that could be used to train a classifier head using a small dataset.

We used a MLP classifier head that used a `linear[1024]-> batchnorm[512] -> GELU-> dropout[0.3]-> linear layer[100]`. We use batchnorm for optimization. GELU for smooth nonlinear activation. Dropout for regularization.

We kept the backbone frozen and only trained on the classifier head using k-folds. With a small dataset, fine tuning the backbone risked overfitting. With keeping the backbone frozen, it allowed us to precompute the image embeddings and reuse them across epochs making training fast and computationally inexpensive.

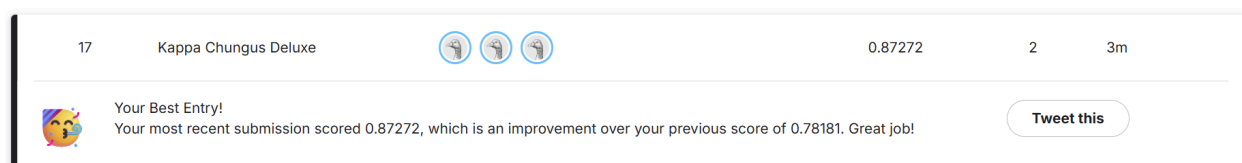
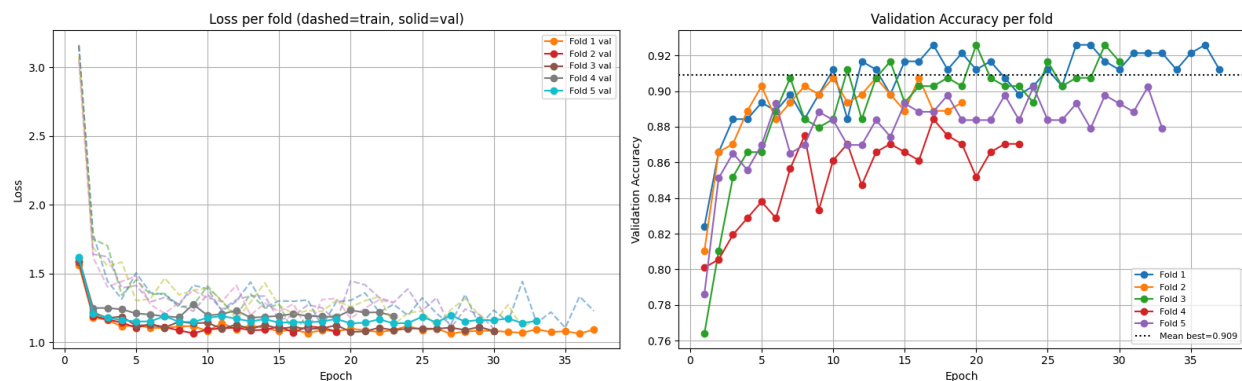
3.2 Training

Our criterion was `SoftTargetCrossEntropy()`, and our optimizer was AdamW. Learning rate is initialized at 0.001, scheduler is CosineAnnealingLR, batch size is 32, 60 epochs, and 0.05 weight decay. Trained on local PC using 5060 8gb VRAM. Pytorch 2.11.0+cu128.

4 Experiments

We started with the basic configuration given in Homework 2 and modified it to ensure correct train, validation, and test sets, and using pretrained weights from EfficientNetV2 instead of a configured CNN model. We kept tuning the parameters, but this took too much time, so we looked into better models that were pretrained. We used K-fold cross validation. We used basic augmentations given in Homework 2 initially, but added standard augmentations that generalized our model to prevent overfitting. Our model size with EfficientNet had 21 million parameters, which is significantly smaller than 300 million parameters in DinoV3. We tried to use progressive unfreezing with EfficientNet, but this made training time painfully slow.

5 Results



6 Discussion

The most apparent improvement was switching to DinoV3 as our pre-trained model. We experimented with EfficientNet, however this model had subpar accuracy figures. Switching to DinoV3 massively increased our test accuracy from 62% to 90%. Using EfficientNetV2 and VisionTransformer suffered low validation accuracy and had overfitting issues. EfficientNetV2

especially suffered from long training times. The biggest limitation was computing power. We trained locally on a personal machine, using hardware credits would massively reduce computation time. One improvement would be to test progressive unfreezing on the backbone to see if there are any increases in accuracy. Also, while DinoV3 proved to be successful, other models could prove to yield better results.

7 Reproducibility

Seed = 42

torch	2.11.0+cu128
torchvision	0.26.0+cu128
transformers	5.9.0
timm	1.0.27
scikit-learn	1.9.0
numpy	2.4.4
pandas	3.0.3
matplotlib	3.10.9
tqdm	4.67.3
pillow	12.2.0
huggingface-hub	1.17.0
safetensors	0.7.0
tokenizers	0.22.2
scipy	1.17.1
python	3.14.0

Request access to DinoV3 on Huggingface. It may take 15-30 minutes to gain access.

Insert Huggingface token in designated section.

Setup Training and test image pathways according to your directory.

After downloading required packages, inserting the hugging face token and setting up training and test paths. Run All.

8 Team Contributions

Robert Schmidling: model selection, training setup, model evaluation

William: parameter tuning, testing, model configuration

Leonard Ifrim: Directory configuration, data preprocessing, data augmentation, data embedding

9 References

[Pretrained model](#)

[DinoV3 License](#)

[DinoV3](#)